“When your only hammer is a keyboard, everything looks like a tool”

Darren Hobbs

Polyglot Programming

An experience report

Who am I ?

Darren Hobbs

Software developer

Language scavenger

Language scavenger

Bash, C#, CoffeeScript, Dart, Erlang, F#, Go, Haskell, Java, Javascript, Lisp, Matlab, Objective-C, OCaml, Perl, Python, PL/SQL, Ruby, Smalltalk, Scheme, TCL…

Language scavenger

Hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer, hammer…

The perfect tool?

“When your only tool is a hammer, everything looks like a thumb”

Programmer’s hammer

What's this talk about?

Polyglot programming

How we made technology choices

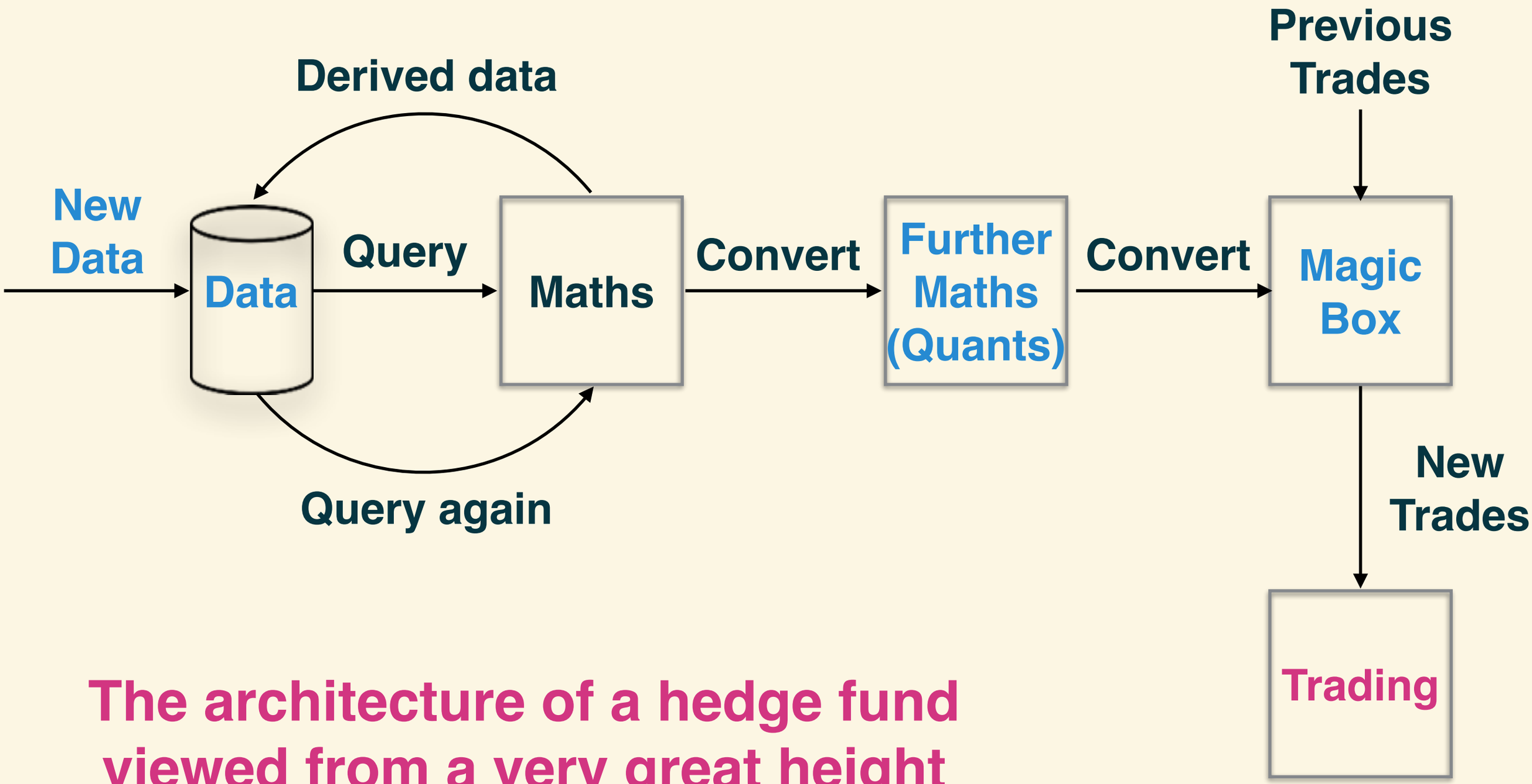
What we learned

What we would do differently

What’s this talk about?

A story about what I did this summer (and autumn (and winter))…

What are we building?



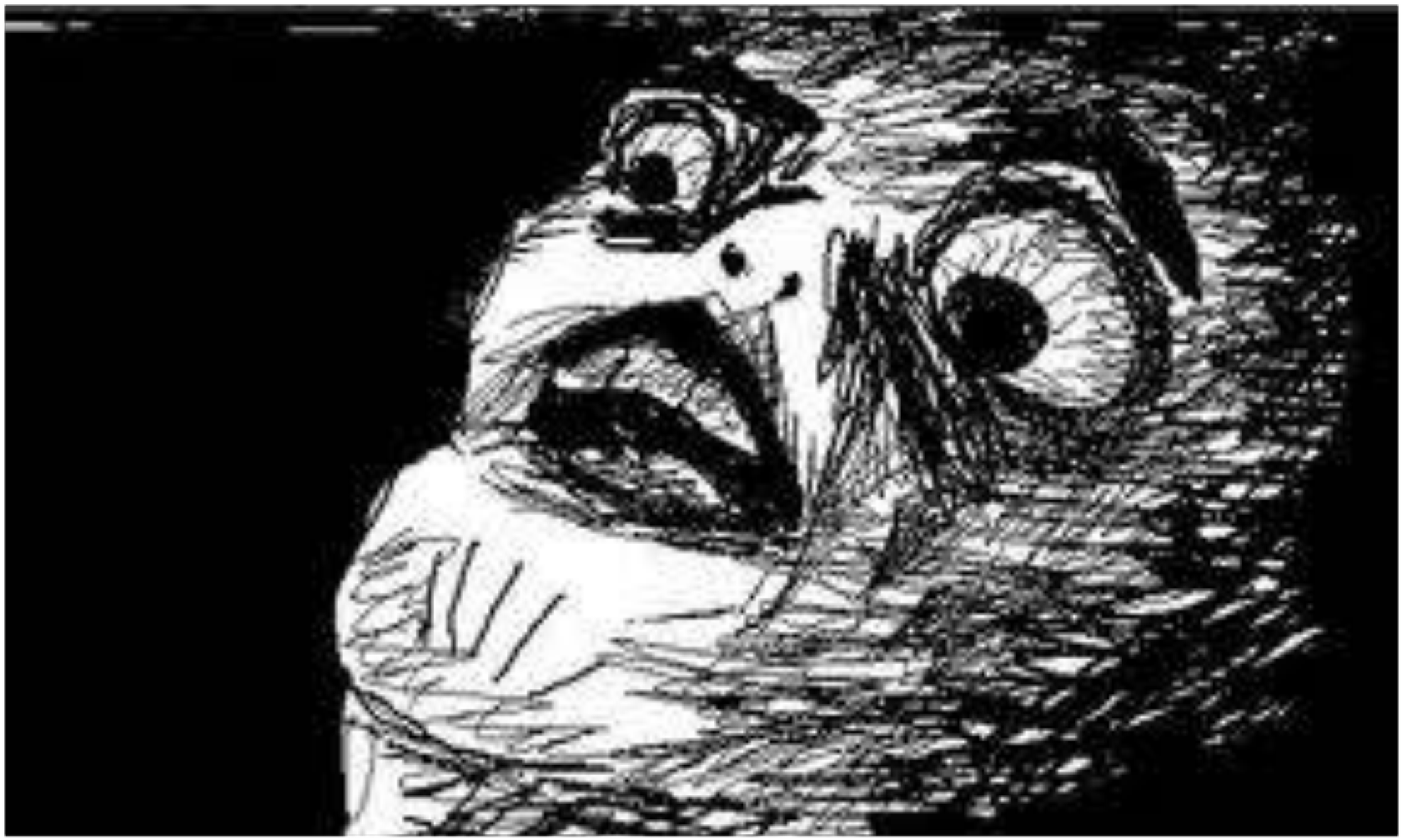
The architecture of a hedge fund viewed from a very great height

What are we building?

A hedge fund.

From scratch.

There’s 3 of us.



We’re going to need a bigger hammer

We need to pick the right tools for the job

What are the right tools?

We don’t know what we don’t know

Defer decisions to the last responsible moment

Do the risky things first

Where are the risks?

Risk: Integration

Risk: Third-party suppliers

Risk 1: Integration with third party suppliers!

Risk 2: Unknown unknowns

We’re going to make mistakes

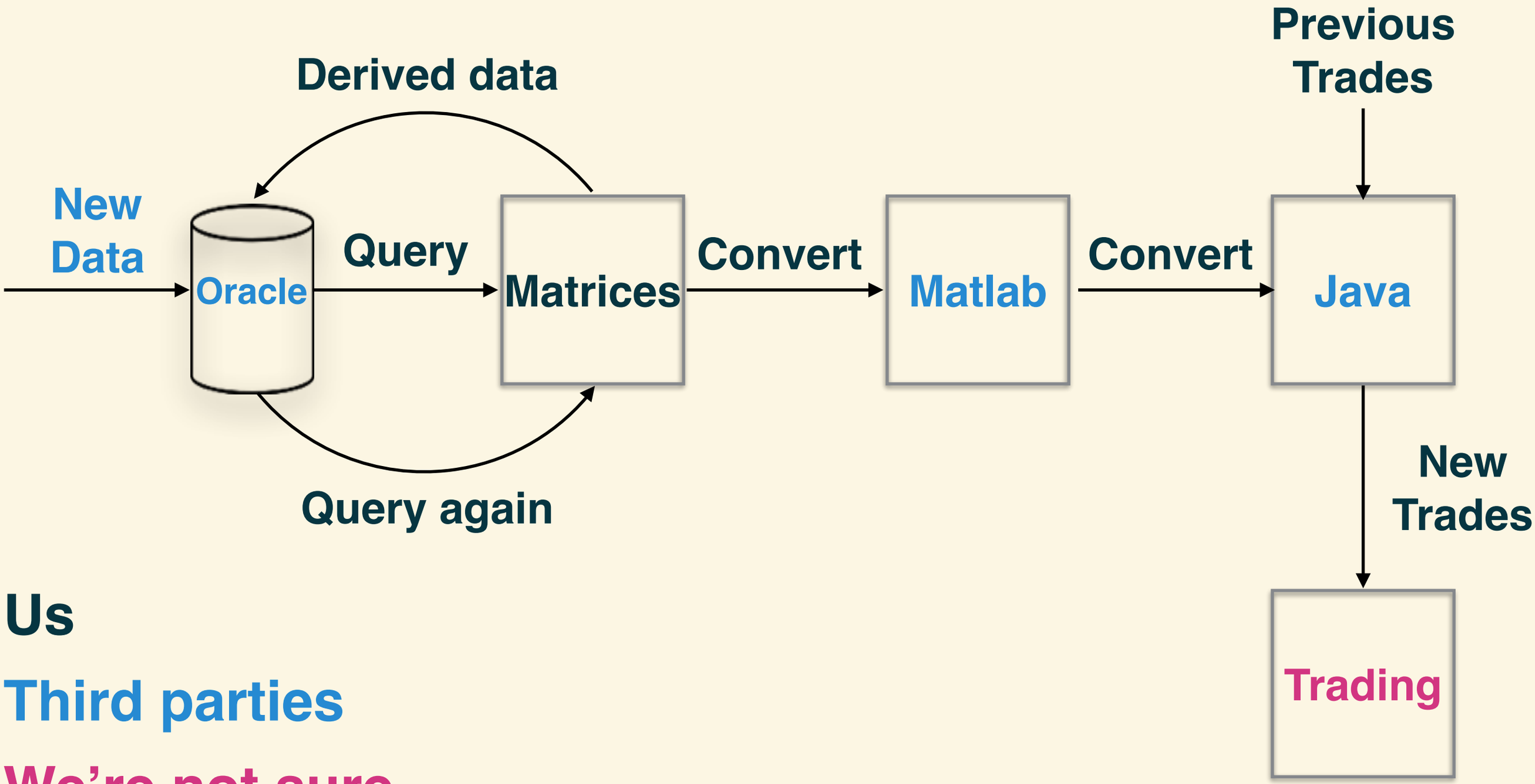
How do we reduce the impact of mistakes?

Make decisions easy to reverse

How do we make decisions easy to reverse?

Keep the pieces small

Designed with risk in mind



Us

Third parties

We're not sure

So what's this got to do with polyglot programming?

Analysing the 3rd party systems told us our technical constraints

Small pieces with well defined roles offer maximum language choice

Time for a training montage

Much googling later…

We considered:

Python

Well established. Good story for mathematical processing using numpy and pandas.

Java

Well established. We have to use it anyway to integrate with the magic box, has libraries for matlab files.

We considered:

Clojure

We considered:

Solid java interop, good knowledge of it on the team (and we like functional programming).

We considered:

Node

Great for quickly building web services, no real benefit over clojure for us.

We considered:

Matlab

Licenses expensive but we have to integrate with it somehow, the quants use it.

We considered:

Octave

Open source version of Matlab. Might be a cheaper alternative.

.net

We considered:

Lots to like about the .net platform itself. Windows unproductive but we might have to use it for the trading system.

Julia

We considered:

Just a little too new and unproven. We're keeping an eye on it!

We considered:

Storm, Hadoop, Cascalog

“Big Data”

Very powerful. Steep learning curve. Overkill for our current needs, might be a good fit in future.

Go

We considered:

Bit new, no knowledge in the team, no benefits over java or clojure for us.

We considered:

Haskell

“I try not to use languages where the compiler has a higher IQ than I do”

PL/SQL

But not for very long.

We considered:

What did we decide?

Pieces of Polyglot

Database: Oracle*

Query: Python / SQL Alchemy

Maths: Python / Pandas

Further maths: Matlab*

Magic box: Java*

Trading: .net* or Java (we're still not sure)

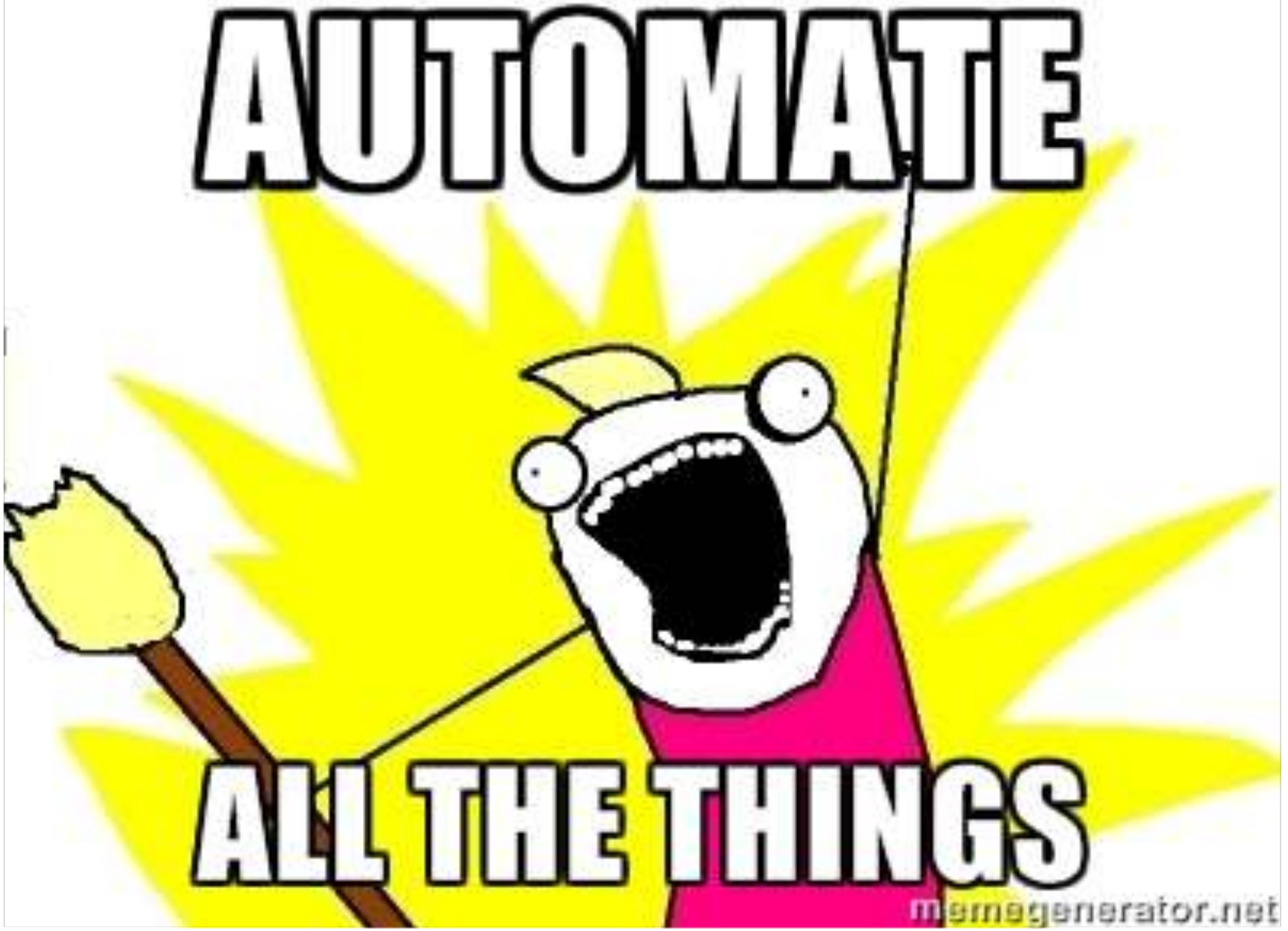
Platform: AWS EC2, S3, SNS/SQS, RDS, VPC

*Constrained by outside forces

Sorted, right?

Here’s what we learned

Small pieces mean more to deploy


AUTOMATE
ALL THE THINGS
memegenerator.net

Automation options

We like:

- Docker: http://www.docker.io
- CoreOS: http://coreos.com
- EC2: http://aws.amazon.com/ec2
- Upstart: http://upstart.ubuntu.com
- Cloud-init: https://help.ubuntu.com/community/CloudInit

Not so much:

- Chef: http://www.opscode.com/chef/
- Puppet: http://puppetlabs.com
- Ansible: http://www.ansibleworks.com

Integration is important

Every new language multiplies integration costs

Multiple languages +
evolving API
= duplication of effort

Make sure the benefits outweigh the costs

We decided to keep it really simple

JSON / CSV / HTTP

We’re still not 100% happy with the integration costs

What else?

Assume nothing!

Then assume that there's an assumption you've made and forgotten about

Assumption: Python deployment

"Python's been around for ages, it's bound to be easy to deploy!"

setuptools

Python deployment

setuptools

distribute

Python

deployment

setuptools
distribute
virtualenv

Python
deployment

Python deployment

setuptools
distribute
virtualenv
pip

Python deployment

setuptools
distribute
virtualenv
pip
easy_install

Python deployment

setuptools
distribute
virtualenv
pip
easy_install
rpm

Python deployment

setuptools
distribute
virtualenv
pip
easy_install
rpm
wtf?

Assumption: Bottleneck

"We need pandas for fast number crunching & matrix maths"

Actual Bottleneck

DB queries are over 90% of the execution time

Was Python a mistake?

Not exactly…

It was a less useful form of success

Great for prototyping

Not so great in production

(We're living with it but not adding more code)

What are we now doing differently?

Migrating from python to clojure

Wrapping the java APIs in clojure

Why clojure?

All the power of a dynamic language, all the resources of the JVM

Team knowledge

```
lein uberjar
```

Was the polyglot thing a waste of time?

Absolutely not!

We’re ‘standardising’ on clojure… (for now!)

Our design means we can still change our minds

One last thing

How did we persuade management?

Vocabulary: risk, options, experimentation, learning

What about hiring?

Options and Risk

Chris Matts & Olav Maassen: http://commitment-thebook.com/

http://www.infoq.com/articles/real-options-enhance-agility

Liz Keogh: http://lizkeogh.com/2013/07/21/estimating-complexity

Thank you!

@darrenhobbs